# Lesson 12 Notes: Toeplitz Matrices

## 1. Main idea of Lesson 12

Lesson 12 teaches **Toeplitz matrices**.

In earlier lessons, we stored special matrices by avoiding unnecessary values:

| Lessons | Matrix type | Main idea |
|---|---|---|
| Lessons 1–4 | Diagonal matrix | Store only positions where `i == j`. |
| Lessons 5–8 | Lower triangular matrix | Store positions where `i >= j`. |
| Lesson 9 | Upper triangular matrix | Store positions where `i <= j`. |
| Lesson 10 | Symmetric matrix | Store one half and use the mirror rule `M[i][j] == M[j][i]`. |
| Lesson 11 | Tridiagonal / band matrix | Store values close to the main diagonal using `i - j`. |
| Lesson 12 | Toeplitz matrix | Store repeated diagonals using the first row and first column. |

The main idea of Lesson 12 is this:

A Toeplitz matrix has repeated values along each diagonal that goes from **top-left to bottom-right**.

Because those diagonal values repeat, we do not need to store the full matrix.

Instead, we store only:

```
first row + first column without repeating the top-left value
```

For an `N x N` Toeplitz matrix, this uses:

```
2N - 1 values
```

instead of:

```
N² values
```

---

## 2. What you should already know before Lesson 12

Before learning Toeplitz matrices, you should understand these ideas:

| Concept | Meaning |
|---|---|
| Matrix | A table of values arranged in rows and columns. |
| Square matrix | A matrix with the same number of rows and columns. |
| Matrix position | A value written as `M[i][j]`, where `i` is the row and `j` is the column. |
| 1-based matrix indexing | Matrix rows and columns usually start from `1` in these lessons. |
| 0-based array indexing | C and C++ arrays start from index `0`. |
| Special matrix | A matrix with a pattern that lets us save memory. |
| 1D array storage | Storing only the needed values in one array. |
| Direct formula | A formula that converts a matrix position `(i, j)` into a 1D array index. |
| Display function | A function that prints the full matrix even though the full matrix is not stored. |

The same pattern from earlier lessons still applies:

1. Understand the matrix pattern.
2. Store only the values needed to recreate the matrix.
3. Use a formula to convert `(i, j)` into a 1D array index.
4. Use `get()` to read a value.
5. Use `set()` to store or change a value.
6. Use `display()` to print the full visible matrix.

---

## 3. What is a Toeplitz matrix?

A **Toeplitz matrix** is a square matrix where each diagonal from **top-left to bottom-right** contains the same value.

Example:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
8 7 6 1 2
9 8 7 6 1
```

This is a `5 x 5` Toeplitz matrix.

Look at the main diagonal:

```
1
  1
    1
      1
        1
```

All values on this diagonal are `1`.

Look at the diagonal above the main diagonal:

```
2
  2
    2
      2
```

All values on this diagonal are `2`.

Look at the diagonal below the main diagonal:

```
6
  6
    6
      6
```

All values on this diagonal are `6`.

So the important rule is:

```
Every top-left to bottom-right diagonal has one repeated value.
```

---

## 4. Toeplitz matrix rule

The formal rule is:

```
M[i][j] == M[i - 1][j - 1]
```

This means:

The value at row `i`, column `j` is the same as the value one step up-left from it.

Example:

```
M[3][4] == M[2][3]
M[2][3] == M[1][2]
```

So all of these positions are on the same diagonal:

```
M[1][2]
M[2][3]
M[3][4]
M[4][5]
```

In the example matrix:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
8 7 6 1 2
9 8 7 6 1
```

Those positions all contain `2`.

---

## 5. Important difference from previous special matrices

Toeplitz matrices are different from diagonal, triangular, and tridiagonal matrices.

In many earlier lessons, we saved memory because many values were structural zeros.

For example:

- A diagonal matrix has zeros outside the main diagonal.
- A lower triangular matrix has zeros above the main diagonal.
- An upper triangular matrix has zeros below the main diagonal.
- A tridiagonal matrix has zeros outside three diagonals.

But a Toeplitz matrix does **not** mainly save memory because of zeros.

A Toeplitz matrix saves memory because values are **repeated along diagonals**.

Example:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
8 7 6 1 2
9 8 7 6 1
```

This matrix has many non-zero values.

But many of those values repeat because of the Toeplitz rule.

So we store only enough values to rebuild all repeated diagonals.

---

## 6. Which values do we store?

For a Toeplitz matrix, every diagonal starts either in:

1. the first row, or
2. the first column.

So we only need to store:

```
first row + first column without repeating the top-left value
```

Using this matrix:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
```

```
8 7 6 1 2
9 8 7 6 1
```

The first row is:

```
1 2 3 4 5
```

The first column is:

```
1
6
7
8
9
```

But the top-left value `1` belongs to both the first row and first column.

We should not store it twice.

So we store:

```
First row: 1 2 3 4 5
First column without top-left: 6 7 8 9
```

Final compact array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Another way to show it:

```
A = [ first row | first column without top-left ]
A = [ 1 2 3 4 5 | 6 7 8 9 ]
```

---

## 7. Storage size formula

A normal `N x N` matrix stores:

```
N² values
```

A Toeplitz matrix stores:

```
first row + first column without repeated top-left
```

The first row has:

```
N values
```

The first column without the top-left value has:

```
N - 1 values
```

So total storage is:

```
N + (N - 1)
```

Simplify:

```
2N - 1
```

So:

```
Toeplitz stored values = 2N - 1
```

---

## 8. Storage count example for N = 5

For a normal `5 x 5` matrix:

```
N² = 5² = 25
```

So normal storage uses `25` values.

For a Toeplitz `5 x 5` matrix:

```
2N - 1 = 2(5) - 1
       = 10 - 1
       = 9
```

So Toeplitz storage uses only `9` values.

Comparison:

| Matrix type | Stored values when N = 5 |
|---|---|
| Normal matrix | 25 |
| Toeplitz matrix | 9 |

So the Toeplitz matrix saves memory by storing only the diagonal patterns.

---

## 9. Compact array structure

For this matrix:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
8 7 6 1 2
9 8 7 6 1
```

The compact array is:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Index view:

```
Index: 0 1 2 3 4 5 6 7 8
Value: 1 2 3 4 5 6 7 8 9
```

The first `N` values come from the first row:

```
A[0] = 1
A[1] = 2
A[2] = 3
```

```
A[3] = 4
A[4] = 5
```

The remaining values come from the first column, excluding the top-left value:

```
A[5] = 6
A[6] = 7
A[7] = 8
A[8] = 9
```

For `N = 5`:

```
A[0] to A[4] store the first row.
A[5] to A[8] store the rest of the first column.
```

---

## 10. Two cases for finding an array index

To get a value from a Toeplitz matrix, we need to know which diagonal the position belongs to.

There are two cases:

| Case | Meaning | Formula |
|---|---|---|
| `i <= j` | Position is on the main diagonal or above it. | `index = j - i` |
| `i > j` | Position is below the main diagonal. | `index = n + (i - j - 1)` |

Here:

```
i = row number
j = column number
n = matrix dimension
```

Important indexing convention:

- Matrix positions use 1-based indexing.
- C/C++ arrays use 0-based indexing.

That is why the formulas produce array indexes starting from `0`.

---

## 11. Case 1: When i <= j

If:

```
i <= j
```

then the position is on the main diagonal or above the main diagonal.

These diagonals start in the first row.

Use this formula:

```
index = j - i
```

### Why does this work?

Example:

```
M[2][4]
```

Here:

```
i = 2
j = 4
```

Since:

```
2 <= 4
```

we use:

```
index = j - i
index = 4 - 2
index = 2
```

So:

```
M[2][4] -> A[2]
```

Using the compact array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

we get:

```
A[2] = 3
```

So:

```
M[2][4] = 3
```

### Visual explanation

Move from `M[2][4]` up-left until you reach the first row:

```
M[2][4] -> M[1][3]
```

So `M[2][4]` uses the same value as the first-row position `M[1][3]`.

Since `M[1][3]` is the third value in the first row, its array index is `2`.

That matches:

```
index = j - i = 4 - 2 = 2
```

---

## 12. Case 2: When i > j

If:

```
i > j
```

then the position is below the main diagonal.

These diagonals start in the first column.

Use this formula:

```
index = n + (i - j - 1)
```

### Why does this work?

Example:

```
M[4][2]
```

Here:

```
n = 5
i = 4
j = 2
```

Since:

```
4 > 2
```

we use:

```
index = n + (i - j - 1)
index = 5 + (4 - 2 - 1)
index = 5 + 1
index = 6
```

So:

```
M[4][2] -> A[6]
```

Using the compact array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

we get:

```
A[6] = 7
```

So:

```
M[4][2] = 7
```

### Visual explanation

Move from `M[4][2]` up-left until you reach the first column:

```
M[4][2] -> M[3][1]
```

So `M[4][2]` uses the same value as first-column position `M[3][1]`.

The first row already uses indexes:

```
A[0] to A[n - 1]
```

For `n = 5`, that means:

```
A[0] to A[4]
```

So the first-column extra values start at:

```
A[5]
```

That is why the lower-position formula begins with `n`:

```
index = n + (i - j - 1)
```

---

## 13. Formula summary

For a Toeplitz matrix stored as:

```
A = [ first row | first column without top-left ]
```

use these formulas:

```
If i <= j:
    index = j - i
```

```
If i > j:
    index = n + (i - j - 1)
```

These formulas directly give the array index in constant time.

---

## 14. Mapping examples

Using:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

and:

```
n = 5
```

we can map matrix positions to array indexes.

### Example 1: M[1][1]

```
i = 1
j = 1
```

Check:

```
1 <= 1
```

True.

Use:

```
index = j - i
index = 1 - 1
index = 0
```

So:

```
M[1][1] -> A[0] -> 1
```

### Example 2: M[1][5]

```
i = 1
j = 5
```

Check:

```
1 <= 5
```

True.

Use:

```
index = j - i
index = 5 - 1
index = 4
```

So:

```
M[1][5] -> A[4] -> 5
```

### Example 3: M[3][3]

```
i = 3
j = 3
```

Check:

```
3 <= 3
```

True.

Use:

```
index = j - i
index = 3 - 3
index = 0
```

So:

```
M[3][3] -> A[0] -> 1
```

This makes sense because the main diagonal repeats the top-left value.

### Example 4: M[5][2]

```
i = 5
j = 2
```

Check:

```
5 <= 2
```

False.

So `i > j`.

Use:

```
index = n + (i - j - 1)
index = 5 + (5 - 2 - 1)
index = 5 + 2
index = 7
```

So:

```
M[5][2] -> A[7] -> 8
```

### Example 5: M[2][1]

```
i = 2
j = 1
```

Check:

```
2 <= 1
```

False.

So `i > j`.

Use:

```
index = n + (i - j - 1)
index = 5 + (2 - 1 - 1)
index = 5 + 0
index = 5
```

So:

```
M[2][1] -> A[5] -> 6
```

---

## 15. Full mapping table for the 5 x 5 example

Matrix:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
8 7 6 1 2
9 8 7 6 1
```

Compact array:

```
Index: 0 1 2 3 4 5 6 7 8
Value: 1 2 3 4 5 6 7 8 9
```

Some useful mappings:

| Matrix position | Case | Formula | Index | Value |
|---|---|---|---|---|
| `M[1][1]` | `i <= j` | `1 - 1` | 0 | 1 |

| Matrix position | Case | Formula | Index | Value |
|---|---|---|---|---|
| `M[1][2]` | `i <= j` | `2 - 1` | 1 | 2 |
| `M[1][5]` | `i <= j` | `5 - 1` | 4 | 5 |
| `M[2][2]` | `i <= j` | `2 - 2` | 0 | 1 |
| `M[2][4]` | `i <= j` | `4 - 2` | 2 | 3 |
| `M[3][3]` | `i <= j` | `3 - 3` | 0 | 1 |
| `M[2][1]` | `i > j` | `5 + (2 - 1 - 1)` | 5 | 6 |
| `M[4][2]` | `i > j` | `5 + (4 - 2 - 1)` | 6 | 7 |
| `M[5][2]` | `i > j` | `5 + (5 - 2 - 1)` | 7 | 8 |
| `M[5][1]` | `i > j` | `5 + (5 - 1 - 1)` | 8 | 9 |

---

## 16. The get operation

The `get` operation returns the value at matrix position `M[i][j]`.

For a Toeplitz matrix:

- If `i <= j`, the value comes from the first-row part of the array.
- If `i > j`, the value comes from the first-column part of the array.

### Code

```
int getToeplitz(int A[], int n, int i, int j) {
    if (i <= j) {
        int index = j - i;
        return A[index];
    } else {
        int index = n + (i - j - 1);
        return A[index];
    }
}
```

### Explanation

The function receives:

```
A = compact Toeplitz array
n = matrix dimension
i = row number
j = column number
```

If:

```
i <= j
```

then the position belongs to a diagonal that starts in the first row.

So the function uses:

```
index = j - i
```

If:

```
i > j
```

then the position belongs to a diagonal that starts in the first column.

So the function uses:

```
index = n + (i - j - 1)
```

Then it returns:

```
A[index]
```

---

## 17. Important point about get()

In many earlier matrix types, `get()` returned `0` for positions that were not stored.

Examples:

- Diagonal matrix: non-diagonal positions return `0`.
- Lower triangular matrix: positions above the diagonal return `0`.
- Upper triangular matrix: positions below the diagonal return `0`.

- Tridiagonal matrix: positions outside the three diagonals return `0`.

But Toeplitz is different.

A Toeplitz matrix usually has real values throughout the matrix.

So `get()` does not return `0` just because the position is not directly stored.

Instead, it finds the correct repeated diagonal value from the compact array.

This is the key difference:

```
Other matrices often use structural zeros.
Toeplitz matrices use repeated diagonal values.
```

---

## 18. Dry run of get(): M[2][4]

Given:

```
n = 5
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Call:

```
getToeplitz(A, 5, 2, 4);
```

Step 1: Identify values.

```
i = 2
j = 4
```

Step 2: Check condition.

```
i <= j
2 <= 4
true
```

Step 3: Use upper/main formula.

```
index = j - i
index = 4 - 2
index = 2
```

Step 4: Return value.

```
return A[2]
```

Since:

```
A[2] = 3
```

Answer:

```
M[2][4] = 3
```

---

## 19. Dry run of get(): M[4][2]

Given:

```
n = 5
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Call:

```
getToeplitz(A, 5, 4, 2);
```

Step 1: Identify values.

```
i = 4
j = 2
```

Step 2: Check condition.

```
i <= j
4 <= 2
false
```

So this position is below the main diagonal.

Step 3: Use lower formula.

```
index = n + (i - j - 1)
index = 5 + (4 - 2 - 1)
index = 5 + 1
index = 6
```

Step 4: Return value.

```
return A[6]
```

Since:

```
A[6] = 7
```

Answer:

```
M[4][2] = 7
```

---

## 20. The set operation

The `set` operation stores a value into the Toeplitz matrix.

For a Toeplitz matrix, setting one matrix position actually changes the value of one whole diagonal.

Why?

Because every value on the same top-left to bottom-right diagonal must be equal.

### Code

```
void setToeplitz(int A[], int n, int i, int j, int x) {
    if (i <= j) {
        int index = j - i;
        A[index] = x;
    } else {
        int index = n + (i - j - 1);
        A[index] = x;
    }
}
```

### Explanation

The function receives:

```
A = compact Toeplitz array
n = matrix dimension
i = row number
j = column number
x = value to store
```

If `i <= j`, it uses:

```
index = j - i
```

If `i > j`, it uses:

```
index = n + (i - j - 1)
```

Then it stores:

```
A[index] = x
```

---

## 21. Important point about set()

In a normal matrix, setting `M[2][4]` changes only one position.

But in a Toeplitz matrix, setting `M[2][4]` changes the stored value for the entire diagonal that contains `M[2][4]`.

Example:

```
setToeplitz(A, 5, 2, 4, 99);
```

This changes the diagonal containing:

```
M[1][3]
M[2][4]
M[3][5]
```

All of those positions must now behave as `99`.

That is correct because a Toeplitz matrix requires all values on the same diagonal to match.

---

## 22. Dry run of set(): set M[2][4] = 99

Given:

```
n = 5
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Call:

```
setToeplitz(A, 5, 2, 4, 99);
```

Step 1: Identify values.

```
i = 2
j = 4
x = 99
```

Step 2: Check condition.

```
i <= j
2 <= 4
true
```

Step 3: Calculate index.

```
index = j - i
index = 4 - 2
index = 2
```

Step 4: Store value.

```
A[2] = 99
```

Before:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

After:

```
A = [1, 2, 99, 4, 5, 6, 7, 8, 9]
```

This changes the diagonal represented by `A[2]`.

---

## 23. The display operation

The compact array stores only `2N - 1` values.

But the user usually wants to see the full `N x N` matrix.

So the display function uses nested loops:

```
for every row i
    for every column j
        print getToeplitz(A, n, i, j)
```

The display function does not store the full matrix.

It only prints the full matrix by using the `getToeplitz()` function for every position.

---

## 24. Full C++ display program

```
#include <iostream>
using namespace std;

int getToeplitz(int A[], int n, int i, int j) {
    if (i <= j) {
        return A[j - i];
    } else {
        return A[n + (i - j - 1)];
    }
}

void displayToeplitz(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << getToeplitz(A, n, i, j) << " ";
        }
        cout << endl;
    }
}

int main() {
    int n = 5;

    // First row: 1 2 3 4 5
    // First column excluding top-left: 6 7 8 9
    int A[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

    displayToeplitz(A, n);

    return 0;
}
```

Output:

```
1 2 3 4 5
6 1 2 3 4
7 6 1 2 3
8 7 6 1 2
9 8 7 6 1
```

## 25. Why display uses nested loops

A `5 x 5` Toeplitz matrix has:

```
5 x 5 = 25 visible positions
```

Even though we store only:

```
2N - 1 = 9 values
```

we still must print all `25` positions to display the full matrix.

So display uses two loops:

```
outer loop = rows
inner loop = columns
```

That is why display takes `O(N²)` time.

---

## 26. Time and space complexity

| Operation | Time | Space | Reason |
|---|---|---|---|
| Storage | — | `O(N)` | Stores only `2N - 1` values. |
| `get()` | `O(1)` | — | Uses one condition and one direct formula. |
| `set()` | `O(1)` | — | Uses one condition and one direct formula. |
| `display()` | `O(N²)` | — | Prints every visible matrix position. |

### Storage complexity

Toeplitz storage uses:

```
2N - 1
```

In Big-O notation, constants and smaller terms are ignored.

So:

```
2N - 1 -> O(N)
```

Therefore, Toeplitz matrix storage is:

```
O(N)
```

### get() complexity

The `get()` function does:

1. one condition check,
2. one formula calculation,
3. one array access.

So:

```
get() = O(1)
```

### set() complexity

The `set()` function does:

1. one condition check,
2. one formula calculation,
3. one array write.

So:

```
set() = O(1)
```

### display() complexity

The display function prints every position in the visible matrix.

For an `N x N` matrix, that is:

```
N² positions
```

So:

```
display() = O(N²)
```

---

## 27. Toeplitz matrix compared with previous matrices

| Matrix type | Why storage is saved | Stored values |
|---|---|---|
| Diagonal | Only main diagonal can be non-zero. | `N` |
| Lower triangular | Values above diagonal are structural zeros. | `N(N + 1) / 2` |
| Upper triangular | Values below diagonal are structural zeros. | `N(N + 1) / 2` |
| Symmetric | One half mirrors the other half. | `N(N + 1) / 2` |
| Tridiagonal | Only three diagonals are useful. | `3N - 2` |
| Toeplitz | Diagonal values repeat. | `2N - 1` |

The Toeplitz matrix is closest in thinking to diagonal-based storage.

But unlike tridiagonal matrices, Toeplitz matrices do not store only a few diagonals.

A Toeplitz matrix has many diagonals, but each diagonal needs only one stored value.

---

## 28. Common beginner mistakes

### Mistake 1: Thinking Toeplitz means mostly zeros

Wrong idea:

```
Toeplitz matrices save space because most values are zero.
```

Correct idea:

```
Toeplitz matrices save space because diagonal values repeat.
```

A Toeplitz matrix may have no zeros at all.

---

### Mistake 2: Storing the top-left value twice

The first row includes the top-left value.

The first column also includes the top-left value.

But we should store it only once.

Correct storage:

```
first row + first column without top-left
```

Wrong storage:

```
first row + full first column
```

If we store the full first column too, we waste one extra cell.

---

### Mistake 3: Using the wrong formula for lower positions

For positions below the main diagonal, do not use:

```
index = j - i
```

because this would become negative.

Example:

```
M[4][2]
index = j - i = 2 - 4 = -2
```

Array index `-2` is invalid.

Correct formula for `i > j`:

```
index = n + (i - j - 1)
```

---

### Mistake 4: Returning 0 for positions not directly stored

In many earlier matrix types, non-stored positions were structural zeros.

But in Toeplitz matrices, non-directly-stored positions are not automatically zero.

They usually have values copied from their repeated diagonal.

So `get()` should calculate the correct repeated diagonal value.

---

## 29. Mental model to remember

Every Toeplitz diagonal goes like this:

```
top-left -> bottom-right
```

Each diagonal has one repeated value.

Each diagonal starts either in:

```
the first row
```

or:

```
the first column
```

So the entire matrix can be rebuilt from:

```
first row + first column without repeating the top-left value
```

That is the simplest way to remember Toeplitz storage.

---

## 30. Final formula sheet

For an `N x N` Toeplitz matrix:

### Storage size

```
stored values = 2N - 1
```

### Compact array

```
A = [ first row | first column without top-left ]
```

### Index formula

```
If i <= j:
    index = j - i

If i > j:
    index = n + (i - j - 1)
```

### get operation

```
Use the formula to find the index.
Return A[index].
```

### set operation

```
Use the formula to find the index.
Store x at A[index].
```

### display operation

```
Loop through all N x N positions.
For each position, call getToeplitz().
Print the returned value.
```

---

## 31. Practice questions

Use:

```
n = 5
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Find the array index and value for each position.

### Question 1

```
M[3][3]
```

Solution:

```
i = 3
j = 3

Since i <= j:
index = j - i
index = 3 - 3
index = 0

A[0] = 1

M[3][3] = 1
```

### Question 2

```
M[1][5]
```

Solution:

```
i = 1
j = 5

Since i <= j:
index = j - i
index = 5 - 1
index = 4

A[4] = 5

M[1][5] = 5
```

### Question 3

```
M[5][2]
```

Solution:

```
i = 5
j = 2
n = 5

Since i > j:
index = n + (i - j - 1)
index = 5 + (5 - 2 - 1)
index = 5 + 2
index = 7

A[7] = 8

M[5][2] = 8
```

### Question 4

```
M[2][1]
```

Solution:

```
i = 2
j = 1
n = 5

Since i > j:
index = n + (i - j - 1)
index = 5 + (2 - 1 - 1)
index = 5 + 0
index = 5

A[5] = 6

M[2][1] = 6
```

### Question 5

```
M[2][5]
```

Solution:

```
i = 2
j = 5
```

```
Since i <= j:
index = j - i
index = 5 - 2
index = 3

A[3] = 4

M[2][5] = 4
```

---

## 32. Exit criteria

You understand Lesson 12 if you can answer these questions:

1. What is a Toeplitz matrix?
2. Why do Toeplitz matrices save memory?
3. Why do we store the first row and first column?
4. Why do we avoid storing the top-left value twice?
5. What is the storage formula for an `N x N` Toeplitz matrix?
6. What formula is used when `i <= j`?
7. What formula is used when `i > j`?
8. Why does `get()` usually not return `0` for non-directly-stored positions?
9. What happens when `set()` changes one diagonal value?
10. Why does display still take `O(N²)` time?

---

## 33. One-line summary

A Toeplitz matrix stores repeated top-left to bottom-right diagonals, so the whole matrix can be represented using only the first row and the first column without repeating the top-left value.